CPU Scheduling Simulator

Jonathan Koral

Contact Information

Team Member:

Jonathan Koral,

jon.koral1214@gmail.com or JONATHAN.KORAL04@bcmail.cuny.edu

Supervisor:

Minh Le,

minh.le@brooklyn.cuny.edu

Abstract

This project is an online CPU Scheduling Algorithm Simulator made to give students a simple way to understand how scheduling algorithms actually work. Students studying CPU scheduling algorithms struggle to verify whether the Gantt charts they wrote down are actually correct. Students who try writing out Gantt charts for the first time may not realize or understand what they are doing wrong. The resources students are left with to compare their answers too are diagrams from textbooks, class notes, or online videos. These resources help show you how to write your gantt chart but it doesn't have the personalization to check if the student's solution is actually correct. With all the different scheduling algorithms the student has to learn, they might get overwhelmed and confused with the different ways of approaching each algorithm. The goal is to create an easy way for anyone interested to visualize and play around with different CPU scheduling algorithms so they can strengthen their understanding of the topic.

Tools

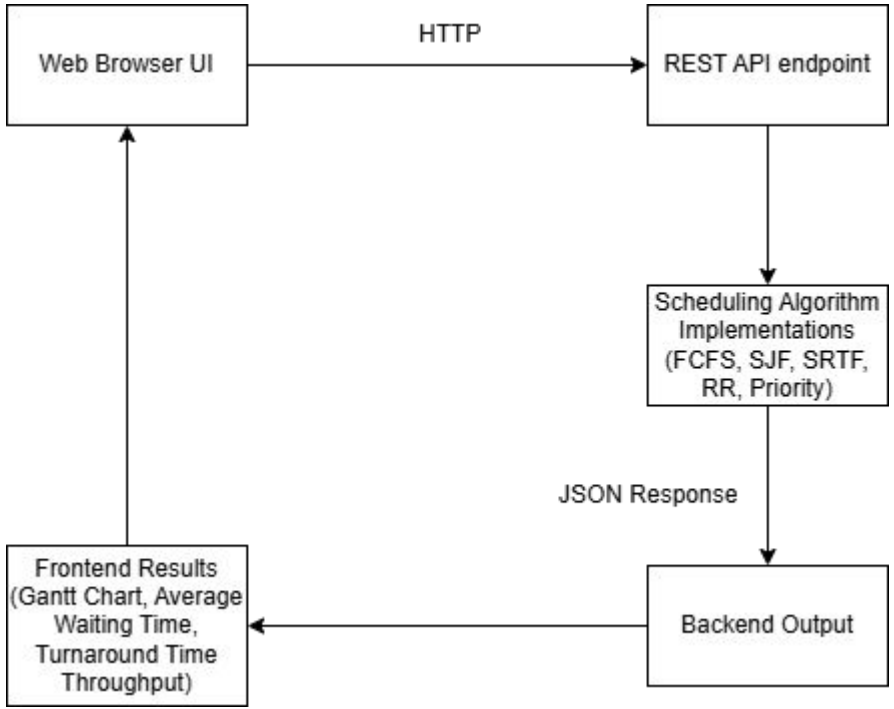
Software:

- Java 25
- Spring Boot 4.0.0
- IntelliJ IDEA
- Github
- HTML, CSS, JS

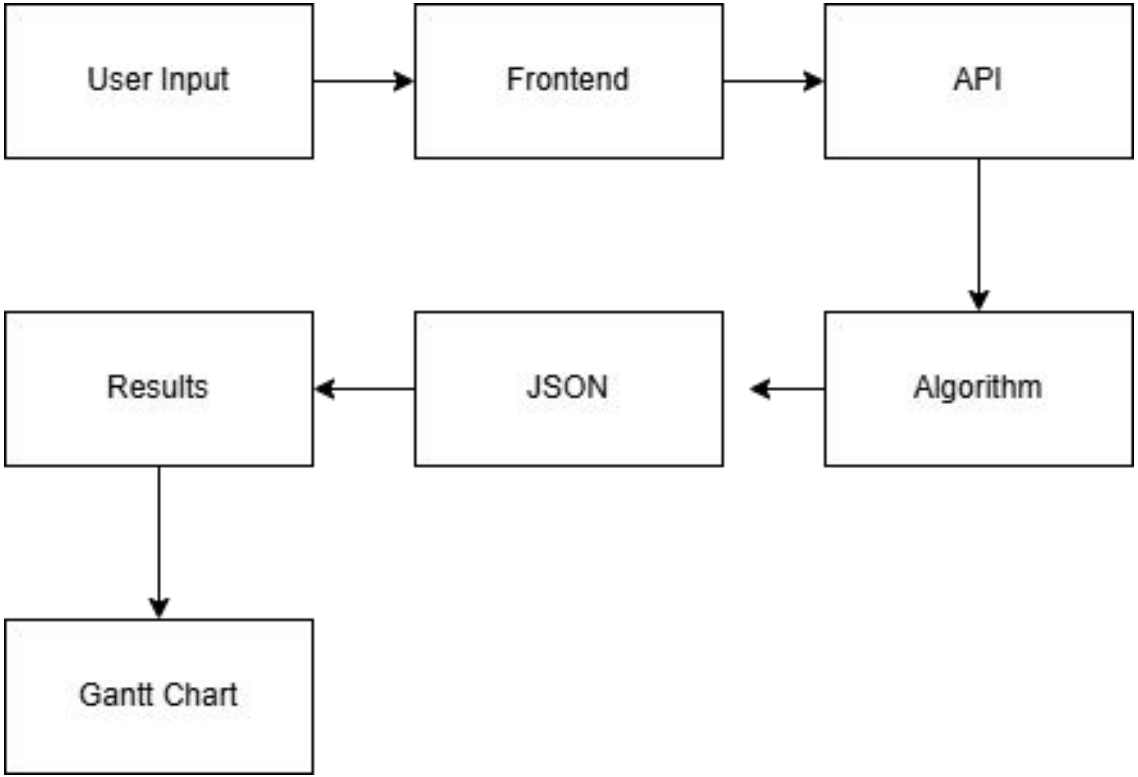
Testing:

- Postman
- JUnit
- Web Browser

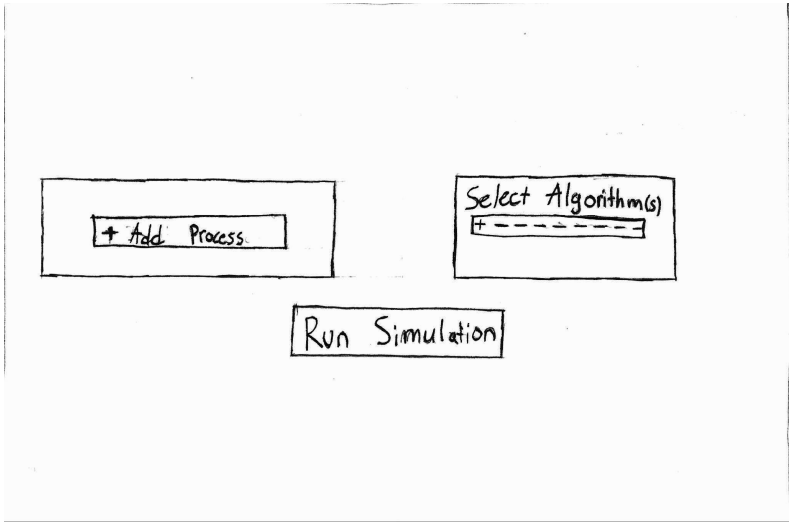
System Architecture

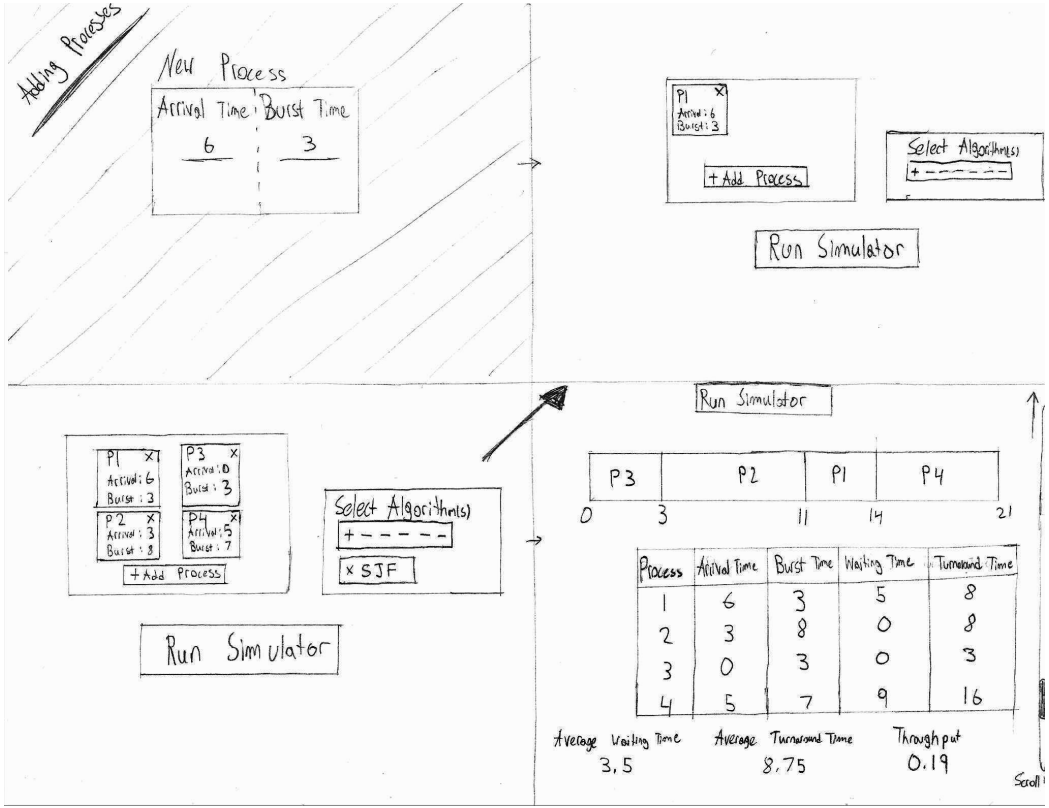


Data Flow Diagram

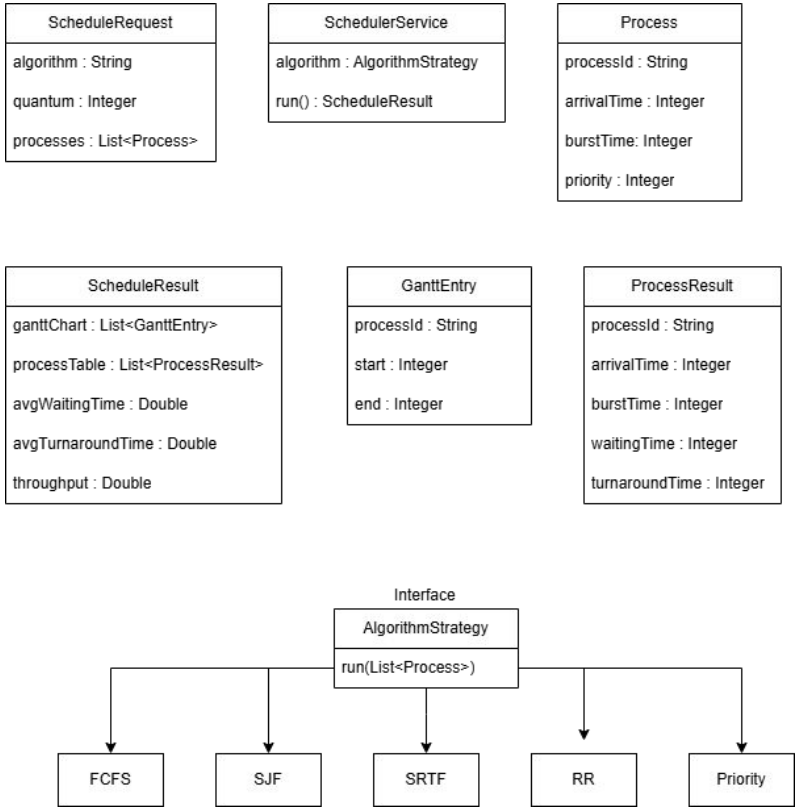


Wireframe

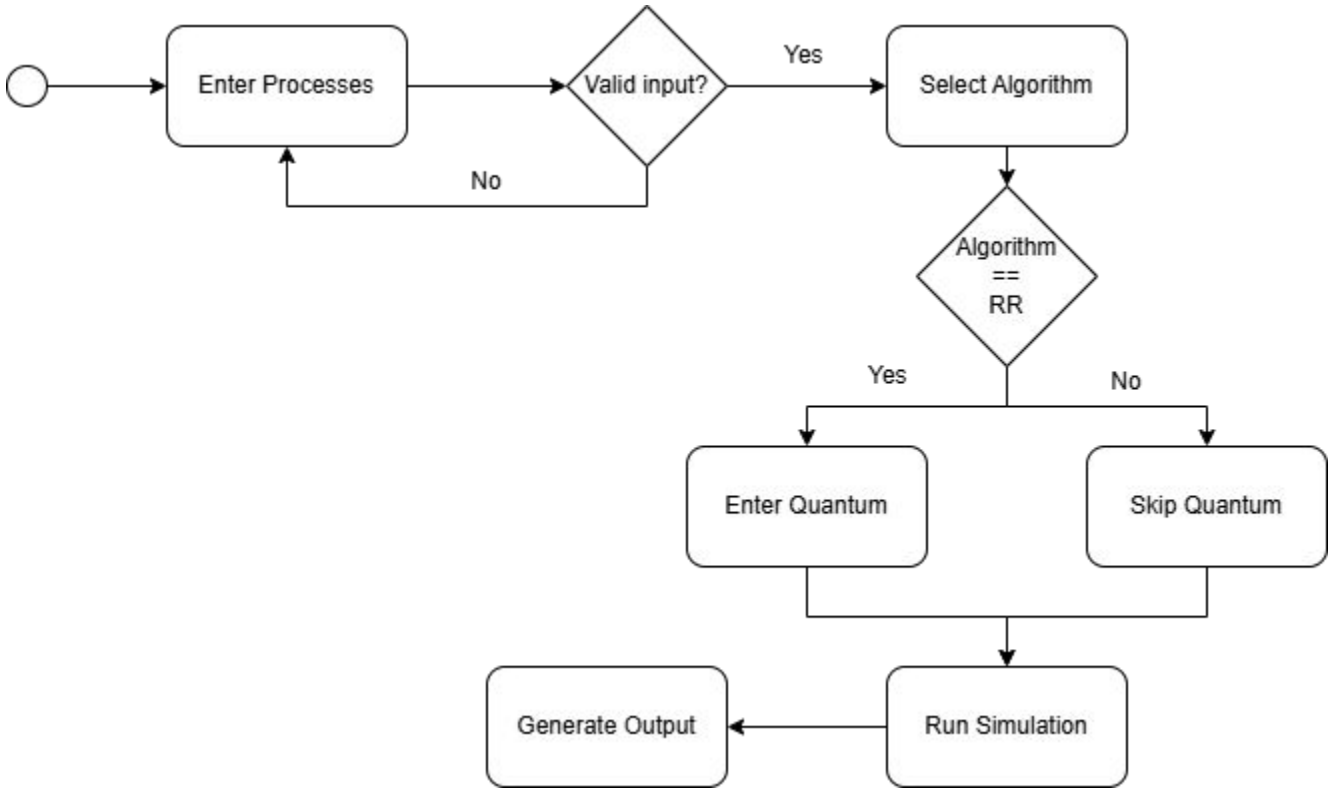

+ Add Process
Select Algorithm(s)
+
Run Simulation


Adding Processes
New Process
Arrival Time
Burst Time
6
3
P1
Arrival: 6
Burst: 3
+ Add Process
Select Algorithm(s)
+
Run Simulator
P1
Arrival: 6
Burst: 3
P3
Arrival: 0
Burst: 3
P2
Arrival: 3
Burst: 8
P4
Arrival: 5
Burst: 7
+ Add Process
Select Algorithm(s)
+
x SJF
Run Simulator
Run Simulator
P3
P2
P1
P4
0
3
11
14
21
Process | Arrival Time | Burst Time | Waiting Time | Turnaround Time
1 | 6 | 3 | 5 | 8
2 | 3 | 8 | 0 | 8
3 | 0 | 3 | 0 | 3
4 | 5 | 7 | 9 | 16
Average Waiting Time
3.5
Average Turnaround Time
8.75
Throughput
0.19

Class Diagrams



Activity Diagram



Tentative Schedule

Week 5	Set up the Backend ● Create the Spring Boot project ● Set up the folder structure ● Create the classes ● Create the REST endpoints	4-5 hours
Week 6	Implement FCFS ● Test the algorithm with a set of inputs Create the Demo Recording video	2-6 hours 5 minutes
Week 7	Implement SJF ● Test the algorithm with a set of inputs	2-5 hours
Week 8	Set up the Frontend ● Create the user input forms ● Add dropdown menu for selecting algorithms ● Test logic and validate user inputs Peer Evaluation	4-8 hours 1 ~ hour(s)

Tentative Schedule

Week 9	Implement SRTF ● Test the algorithm with a set of inputs	2-5 hours
Week 10	Work on Frontend ● Fixing up the style for the Gantt chart ● Make the interface more user friendly ● Update the design (layout/colors)	4-8 hours
Week 11	Finish up the full draft of the slides ● Fix up slides and add whatever is required	15 ~ hours (total work after 1st draft)
Week 12	Implement Round Robin ● Update the frontend to accept quantum time when Round Robin is the selected algorithm. ● Test the algorithm with a set of inputs	2-6 hours

Tentative Schedule

Week 13	Create the second Demo Recording video Implement Priority Scheduling ● Update frontend to accept priority as input for processes ● Test the algorithm with a set of inputs	5 minutes 2-5 hours
Week 14	Final work on Frontend ● Update the user interface ● Make any improvements on the layout ● Implement pop up error messages	4-7 hours
Week 15	Final Slides ● Strengthen the slides that need work ● Add new material as required Create the Final Demo Recording video Submit the project to be graded	5-7 hours 5 minutes 1 minute

Use Case 1 : Student Checks Homework

A student uses my CPU scheduling simulator to verify their homework solutions.

Student enters the following processes to the simulator:

- P1: arrival 0, burst 5
- P2: arrival 2, burst 3
- P3: arrival 4, burst 1

The student selects a scheduling algorithm and runs the simulation.

The output the student receives:

- Gantt Chart
- Average Waiting Time
- Turnaround Time
- Throughput

Use Case 2 : Comparing Algorithms

The user wants to understand how different algorithms behave with the same input.

The user enters the following processes to the simulator:

- P1: arrival 3, burst 4
- P2: arrival 6, burst 3
- P3: arrival 8, burst 2

The user selects the algorithms FCFS, SRTF, RR(Q=2) and runs the simulation.

The output the user receives is different Gantt charts, waiting/turnaround times, and throughput values.

The user can now easily compare and contrast the different algorithms and how they respond to the same processes.

Use Case 3 : Instructor Demonstrations

An instructor wants a way to show the class how Priority scheduling works.

The instructor enters the following processes to the simulator:

- P1: arrival 0, burst 6, priority 3
- P2: arrival 1, burst 4, priority 1
- P3: arrival 2, brust 5, priority 2

The instructor selects the Priority scheduling algorithm and runs the simulation.

The instructor receives the output of a gantt chart showing P2 | P3 | P1, along with the values of the waiting/turnaround time, and throughput.

The instructor can now show the class in real time, how priority scheduling works along with visualizing the pros and cons.